

CERTIFICACIÓN UNIVERSITARIA



UNC



FCEyN



Manual para el alumno

Proyecto
Integrador Final
(PIN)



**WE WANT
SKILLS!**

mundosE



UNC



FCEyN

mundosE

Manual para el alumno: Proyecto Integral Final (PIN)	2
Objetivo general del PIN:.....	2
Consigna.....	2
Contenido de cada proyecto práctico:.....	2
Proyecto 1: CI/CD con GitHub Actions + Terraform + Docker	3
Rúbrica Proyecto 1.....	3
Proyecto 2: CI/CD con GitLab CI + OpenTofu + Kubernetes	4
Rúbrica Proyecto 2.....	5
Proyecto 3: Build & Package + Seguridad en GitHub	5
Rúbrica Proyecto 3.....	6
Proyecto 4: Monitoreo + Seguridad en GitLab	6
Rúbrica Proyecto 4.....	7
Respecto a la evaluación:.....	7
¿Cómo interpretar tu rúbrica de trabajo?.....	7

Manual para el alumno: Proyecto Integral Final (PIN)

Objetivo general del PIN:

Que los estudiantes integren los contenidos vistos en la diplomatura mediante proyectos prácticos que combinen IaC, CI/CD, contenedores, seguridad y monitoreo, aplicando buenas prácticas de trazabilidad y entrega continua.

Consigna

Los estudiantes, organizados en equipos, desarrollan los proyectos prácticos descritos en este documento. A efectos de la presentación final, eligen un (1) proyecto, suben el archivo con la documentación (entregables) al drive “PIN” y lo presentan en el encuentro final como requisito para obtener la certificación del programa.

Cada proyecto práctico incluye:

- Objetivo.
- Herramientas a usar (una por categoría, según el programa)
- Entregables claros.
- Opción local y nube.
- Rúbrica de evaluación detallada.

Contenido de cada proyecto práctico:

- ✓ Una sola herramienta por categoría (según lo visto en el curso).
- ✓ Opción local y opción nube en cada caso.
- ✓ Seguridad siempre incluida.
- ✓ Aplicación base simple generada con IA (como en el encuentro de IA en DevOps).
- ✓ Rúbrica específica por proyecto práctico.

Proyecto 1: CI/CD con GitHub Actions + Terraform + Docker

Objetivo

Construir un pipeline en **GitHub Actions** que compile, teste y despliegue una aplicación generada con IA en un contenedor Docker. La infraestructura debe ser gestionada con **Terraform**. Incluir controles de seguridad en el pipeline.

Herramientas

- CI/CD: GitHub Actions
- IaC: Terraform
- Contenedores: Docker
- Seguridad: SonarQube/ESLint + Snyk
- Monitoreo: Prometheus + Grafana

Entregables

Presentar en un archivo comprimido (.zip o .tar.gz) los siguientes elementos:

- Workflow **.yaml** en GitHub Actions.
- Archivos Terraform (**.tf**) para levantar infraestructura.
- **Dockerfile** y artefacto generado.
- SBOM (CycloneDX/SPDX).
- Captura del dashboard de métricas básicas.

Nombrar el archivo comprimido (ejemplo: Proyecto 1_EquipoX.zip).

Opción local: Docker + Terraform en VirtualBox.

Opción nube: AWS (si se desea cambiar, consultar al docente).

Rúbrica Proyecto 1

Criterio	Descripción	Aporte al proyecto
Pipeline CI/CD	Workflow ejecuta build, tests y despliegue correctamente	25%
Infraestructura	Terraform despliega entorno local/nube correctamente	20%
Contenedor	Imagen Docker reproducible, con Dockerfile documentado	15%

Seguridad	SBOM + análisis de código/dependencias en pipeline	20%
Observabilidad	Dashboard en Prometheus/Grafana con métricas visibles	10%
Documentación	README claro + capturas/video demostrativo	10%

Proyecto 2: CI/CD con GitLab CI + OpenTofu + Kubernetes

Objetivo

Configurar un pipeline en **GitLab CI** que construya y despliegue una aplicación generada con IA en un clúster **Kubernetes**. La infraestructura debe ser gestionada con **OpenTofu**. Incorporar controles de seguridad en el flujo.

Herramientas

- CI/CD: GitLab CI
- IaC: OpenTofu
- Contenedores: Kubernetes (con Minikube o nube)
- Seguridad: Trivy + OWASP Dependency-Check
- Monitoreo: Prometheus + Grafana

Entregables

Presentar en un archivo comprimido (.zip o .tar.gz) los siguientes elementos:

- `.gitlab-ci.yml` funcional.
- Manifiestos `.tf` de OpenTofu.
- `deployment.yaml` y `service.yaml`.
- Imagen firmada con cosign.
- Dashboard de métricas exportado.

Nombrar el archivo comprimido (ejemplo: Proyecto 2_EquipoX.zip).

Opción local: Minikube + OpenTofu.

Opción nube: Azure (si se desea cambiar, consultar al docente).

Rúbrica Proyecto 2

Criterio	Descripción	Aporte al proyecto
Pipeline CI/CD	Jobs de build, test y despliegue en GitLab CI	25%
Infraestructura	OpenTofu crea y gestiona recursos correctamente	20%
Kubernetes	Manifiestos correctos, app funcionando en cluster	15%
Seguridad	Imagen firmada + escaneo de dependencias/vulnerabilidades	20%
Monitoreo	Prometheus/Grafana con dashboard exportado	10%
Documentación	README y capturas/video claros	10%

Proyecto 3: Build & Package + Seguridad en GitHub

Objetivo

Generar un artefacto reproducible, empaquetarlo con metadatos, producir un **SBOM**, firmarlo con **cosign** y publicarlo en un repositorio de artefactos. Validar la firma y el SBOM en el pipeline.

Herramientas

- CI/CD: GitHub Actions
- IaC: Terraform
- Contenedores: Docker (para empaquetar)
- Seguridad: cosign + CycloneDX/SPDX
- Repositorio de artefactos: Harbor/Nexus

Entregables

Presentar en un archivo comprimido (.zip o .tar.gz) los siguientes elementos:

- **Dockerfile** y artefacto construido.
- SBOM generado.
- Firma digital con cosign.
- Publicación en Harbor/Nexus.

- Workflow `.yaml` que valide firmas.

Nombrar el archivo comprimido (ejemplo: Proyecto 3_EquipoX.zip)

Opción local: Harbor con Docker Compose.

Opción nube: GCP (si se desea cambiar, consultar al docente).

Rúbrica Proyecto 3

Criterio	Descripción	Aporte al proyecto
Build/Empaquetado	Artefacto reproducible + Dockerfile válido	25%
Seguridad	SBOM completo + firma cosign verificada	25%
Publicación	Artefacto publicado en Harbor/Nexus con logs	20%
IaC	Terraform prepara entorno básico (local/nube)	15%
Documentación	README con pasos, capturas o video	15%

Proyecto 4: Monitoreo + Seguridad en GitLab

Objetivo

Desplegar una aplicación generada con IA y montar un pipeline en **GitLab CI** que incluya despliegue, escaneo de seguridad y monitoreo con **Prometheus/Grafana**. IaC con **Terraform** o **OpenTofu**.

Herramientas

- CI/CD: GitLab CI
- IaC: Terraform u OpenTofu
- Contenedores: Docker
- Seguridad: OWASP ZAP + SonarQube
- Monitoreo: Prometheus + Grafana

Entregables

Presentar en un archivo comprimido (.zip o .tar.gz) los siguientes elementos:

- `.gitlab-ci.yaml` completo.

- Archivos IaC (.tf).
- Configuración Prometheus/Grafana + dashboard.
- Reportes de escaneo OWASP ZAP/SonarQube.

Nombrar el archivo comprimido (ejemplo: Proyecto 4_EquipoX.zip)

Opción local: Docker Compose + Terraform/OpenTofu.

Opción nube: AWS o Azure (si se desea cambiar, consultar al docente).

Rúbrica Proyecto 4

Criterio	Descripción	Aporte al proyecto
Pipeline CI/CD	Jobs de build, despliegue y seguridad en GitLab CI	25%
Infraestructura	IaC (Terraform/OpenTofu) correcto	20%
Seguridad	OWASP ZAP + análisis de calidad en pipeline	20%
Monitoreo	Prometheus/Grafana con dashboard exportado	20%
Documentación	README con instrucciones claras + evidencia	15%

Respecto a la evaluación:

¿Cómo interpretar tu rúbrica de trabajo?

La rúbrica es una guía práctica para organizar el desarrollo de cada proyecto. Cada criterio refleja un aspecto clave que el equipo debe atender. El “aporte al proyecto (%)” indica el peso relativo de cada criterio en la evaluación final: mientras más alto sea el porcentaje, más importante es para lograr un proyecto completo.

¿Cómo usarla?

- Entender el propósito: Cada criterio conecta con las competencias que el programa busca desarrollar (automatización, infraestructura como código, contenedores, seguridad y monitoreo).
- Usar la rúbrica como checklist: A medida que avancen, verifiquen que cada entregable responde a los criterios.

- Diferenciar lo crítico de lo complementario: Aunque todos los criterios suman valor, algunos aportan más peso que otros. Esto no significa que los de menor porcentaje puedan omitirse, pero sí que los mayores deben resolverse con prioridad.

En resumen, la rúbrica es tu mapa de ruta: te muestra qué tiene mayor impacto y cómo balancear el esfuerzo del equipo para llegar bien preparados a la presentación final.